

Parte 2: Triggers

Para cada gatilho, justificativa do momento (BEFORE/AFTER) e do uso de NEW/OLD.

trg_valida_saldo_before_update

- **Momento (BEFORE):** Executado antes de alterar a tabela para validar a regra de negócio e barrar a gravação caso o saldo fique negativo, economizando recursos do banco.
- **NEW / OLD:** Utiliza NEW.saldo para inspecionar o valor atualizado que está prestes a ser gravado e aplicar a validação.

trg_bloqueia_update_historico

- **Momento (BEFORE):** Intercepta o comando antes da alteração para validar o estado do registro e cancelar a operação caso o histórico já esteja concluído.
- **NEW / OLD:** Utiliza OLD.concluido para verificar o estado original do registro antes da tentativa de modificação.

trg_bloqueia_delete_historico

- **Momento (BEFORE):** Intercepta a tentativa de exclusão antes que a linha seja removida do banco, abortando o comando imediatamente.
- **NEW / OLD:** Não utiliza NEW nem OLD, pois o objetivo é simplesmente bloquear qualquer instrução DELETE na tabela.

trg_auditoria_perfis_update

- **Momento (AFTER):** Executado após a alteração ser confirmada na tabela, garantindo que o log refira-se a uma modificação bem-sucedida.
- **NEW / OLD:** Utiliza OLD para registrar os valores anteriores (OLD.nome_exibicao, OLD.ativo) e NEW para registrar os novos valores aplicados (NEW.nome_exibicao, NEW.ativo).

trg_saneamento_assinante_before_insert

- **Momento (BEFORE):** Executado antes da inserção para transformar e padronizar o dado antes de sua gravação definitiva.
- **NEW / OLD:** Utiliza NEW.nome_completo para ler a string enviada, aplicar o saneamento (UPPER e TRIM) e sobrescrever o valor a ser gravado.

Comparação entre Gatilho (BEFORE UPDATE) e Restrição CHECK para a RN01

A validação da regra de negócio de saldo não negativo (RN01) pode ser abordada tanto por uma restrição CHECK quanto por um gatilho BEFORE UPDATE, mas ambas apresentam comportamentos e propósitos distintos no banco de dados. A restrição CHECK atua de forma declarativa diretamente na estrutura da tabela, garantindo que qualquer valor inserido ou atualizado respeite a condição $\text{saldo} \geq 0$. Caso uma instrução tente violar essa regra, o motor do SGBD interrompe a operação e lança um erro de integridade nativo (SQLSTATE '23000'). Apesar de sua altíssima performance por ser executada no nível do motor de armazenamento, a restrição CHECK oferece pouca flexibilidade na personalização de mensagens para a aplicação e limita-se a avaliar a própria linha no momento da gravação.

Por outro lado, o gatilho `trg_valida_saldo_before_update` foi a opção escolhida para a implementação da RN01 por oferecer maior controle sobre as regras de negócio. O gatilho intercepta a tentativa de alteração antes que ela seja gravada e utiliza a pseudo-variável `NEW.saldo` para inspecionar o valor futuro. Caso o novo saldo seja menor que zero, o gatilho cancela a atualização e dispara uma exceção customizada via `SIGNAL SQLSTATE '45000'` com a mensagem legível *"Operacao cancelada: Saldo do assinante nao pode ficar negativo"*. Embora o gatilho adicione um pequeno custo computacional por ser interpretado a cada UPDATE, ele permite entregar um retorno claro para a camada da aplicação, além de possibilitar comparações diretas com o estado anterior (`OLD.saldo`) ou consultas a tabelas auxiliares.

Testes das Procedures e Resultados Obtidos

-- 1.1 chama a procedure para cadastrar um novo assinante

```
CALL criar_assinantes(  
    'Cesar Pires Lopes',  
    '98765432100',  
    'cesarpl@gmail.com',  
    '2002-11-09',  
    'SP'  
);
```

Chamada da procedure de cadastro de um novo assinante (todas as procedures estão presentes no streamflow.sql e estão sendo usadas no app.sql)

103 11:36:36 CALL criar_assinantes('Cesar Pires Lopes', '98765432100', 'cesarpl@gmail.com', '2002-11-09', 'SP') 1 row(s) affected

Resultado obtido

-- 1.2 recupera dados de um assinante

```
CALL informacoes_assinantes(1);
```

Procedure de listar os assinantes atuais

	id	nome_completo	email	cpf	data_nascimento	uf	saldo	data_cadastro
▶	1	CESAR PIRES LOPES	cesarpl@gmail.com	98765432100	2002-11-09	SP	0.00	2026-09-15 11:36:36

Resultado obtido

-- 1.3 adiciona crédito na conta de um assinante

```
CALL inserir_saldo(1, 150.00);
```

Procedure para inserir saldo na conta, a fim de pagar a mensalidade

	id	nome_completo	email	cpf	data_nascimento	uf	saldo	data_cadastro
▶	1	CESAR PIRES LOPES	cesarpl@gmail.com	98765432100	2002-11-09	SP	150.00	2026-09-15 11:36:36

Resultado obtido

-- 1.4 cobrança da mensalidade

```
CALL assinatura(1);
```

Procedure de cobrança de mensalidade

	id	nome_completo	email	cpf	data_nascimento	uf	saldo	data_cadastro
▶	1	CESAR PIRES LOPES	cesarpl@gmail.com	98765432100	2002-11-09	SP	110.10	2026-09-15 11:36:36

Resultado obtido

```
-- 1.5 atualizar dados cadastrais (para, por exemplo, mudar o UF)
CALL atualizar_dados_assinantes(
  1,
  'Cesar Pires Lopes',
  '98765432100',
  'cesarpl@gmail.com',
  '2002-11-09',
  'MG'
);
```

Procedure que atualiza os dados cadastrais (nesse caso, o UF)

	id	nome_completo	email	cpf	data_nascimento	uf	saldo	data_cadastro
▶	1	Cesar Pires Lopes	cesarpl@gmail.com	98765432100	2002-11-09	MG	110.10	2026-09-15 11:36:36

Resultado obtido

```
-- 2.1 cria perfis que são dependentes na conta do assinante
CALL criar_perfis(1, 'João');
CALL criar_perfis(1, 'Espaço Kids');
CALL criar_perfis(1, 'Perfil da Sala');
```

Criação de perfis dependentes do assinante

	id	id_assinante	nome_exibicao	classificacao	ativo	data_criacao
▶	1	1	João	LIVRE	1	2026-09-15 11:37:59
	2	1	Espaço Kids	LIVRE	1	2026-09-15 11:37:59
	3	1	Perfil da Sala	LIVRE	1	2026-09-15 11:37:59

Resultado obtido

```
-- 2.3 renomeia um perfil existente
CALL atualizar_perfis(1, 'João - Séries');
```

Renomeia algum perfil com base no seu id e o novo nome

	id	id_assinante	nome_exibicao	classificacao	ativo	data_criacao
▶	1	1	João - Séries	LIVRE	1	2026-09-15 11:37:59
	2	1	Espaço Kids	LIVRE	1	2026-09-15 11:37:59
	3	1	Perfil da Sala	LIVRE	1	2026-09-15 11:37:59

Resultado obtido


```
-- 3.1 Registrar evento de "Play" em um filme/episódio (Gera Log e Histórico)
CALL registrar_reproducao(1, 1, '189.23.45.12', 'SMARTTV', @log_reproducao_id);
```

```
-- Exibir o ID do log de reprodução gerado na transação
SELECT @log_reproducao_id AS id_evento_log;
```

Registra uma reprodução (primeira linha)

	id	titulo	tempo_assistido_segundos	duracao_seg	data_ultima_atualizacao
▶	1	Trailer - Marty Supreme	0	3600	2026-09-15 11:43:30

(Segunda linha) registra o play em algum vídeo no histórico, dando um id gerado

	id_evento_log
▶	1

Resultado obtido

Declaração do SGBD e Adaptações de Sintaxe

O banco de dados foi implantado e testado no SGBD MariaDB, utilizando o MySQL Workbench como ferramenta cliente de administração. Durante o desenvolvimento, foram necessárias adaptações de sintaxe devido a divergências entre o padrão aceito pelo MySQL e as exigências do MariaDB. A principal alteração ocorreu no comando de atribuição de papéis padrão aos usuários: a palavra-chave TO, utilizada nativamente no MySQL, precisou ser substituída por FOR (sintaxe exigida pelo MariaDB), resultando no formato SET DEFAULT ROLE role FOR 'usuario'@'%'. Adicionalmente, as aspas simples do identificador da role foram removidas na atribuição no MariaDB para evitar falhas de interpretação do servidor.

Outra adequação importante foi realizada nas Stored Procedures para tratar o aviso Warning 1329 (SQLSTATE '02000' - No data - zero rows fetched). No MariaDB, a instrução SELECT INTO dispara um aviso de execução caso a consulta não retorne nenhuma linha, como no caso da busca por gêneros não cadastrados. Para solucionar essa divergência e evitar que alertas de execução chegassem à aplicação, a consulta interna da procedure registrar_preferencias foi ajustada para utilizar a função de agregação SELECT MAX(id) INTO v_id_genero, garantindo o retorno explícito de valor NULL de forma limpa quando nenhum registro for encontrado.